Proceedings of the 1993 IEEE/RSJ International Conference on Intelligent Robots and Systems Yokohama, Japan July 26-30, 1993

An On-line and Deadlock-free Path-planning Algorithm Based on World Topology

Hiroshi Noborio and Takashi Yoshioka
Department of Precision Engineering, Osaka Electro-Communication University
Hatsu-Cho 18-8, Neyagawa, Osaka 572

Abstract — To ensure a deadlock-free character in a sensor-based path-planning algorithm, we should devise its selection rules of a leave point around an obstacle in a 2-d uncertain world. In general, the sensor information is much used for avoiding some closer obstacles locally but is not useful in keeping an arrival of the automaton to the goal globally. Thus each of all previous algorithms prepared an elegant set of selection rules to assure the convergence of the automaton toward the goal. The most popular concept of the assurance is to decrease the Euclidean distance between the leave point and the goal monotonously or asymptotically.

However in general, the automaton allows some self-positioning error and the error frequently makes a mess of the elegant concept. That is, the sensor-based path-planning algorithm may fail in the adequate selection of the leave point under the influence of the error. In this case, the algorithm does a periodic motion in some loop and consequently the automaton does not catch the goal forever. Thus to keep the automaton's convergence in a sensor-based algorithm, we must check if each of its selection rules is to be robust against the position error or not.

In all previous sensor-based algorithms, their selections consist of two or more rules and each of them is individually influenced by the position error. Thus under the position error, the larger the number of rules is, the smaller the possibility of the convergence of the automaton toward the goal is. In addition, the selection always includes a distance rule between the present and the goal points. However the distance rule is especially affected by the self-positioning error. In result, the previous sensor-based path-planning algorithms are not always suitable for path-planning of an automaton with some dead-reckoning error.

Thus in this paper, a new sensor-based algorithm is proposed for path-planning of an automaton in a 2-d world. In the new algorithm, the number of selection rules of the leave point is reduced to one, and also only a rule is not to be any distance rule between the present and the goal points but some topological rule of the world. The topological character is generally to be robust for the self-positioning error. Therefore the new sensor-based path-planning algorithm leads the automaton to the goal in safety even if the automaton generates some position error in the world.

I. Introduction

In the last five years, some sensor-based algorithms have been developed for path-planning of an automaton in a 2-d uncertain world. The common concept of all previous algorithms is as follows: An automaton has some local sensor to find its surrounding obstacles. The automaton basically goes straight to the goal if the local sensor does not react on any touch against some obstacle. Otherwise, that is, if the automaton touches an obstacle by an arbitrary point H, it traces the obstacle until the automaton leaves it from an adequate point L. Then the automaton goes straight to the goal again. The two actions, that is, straight-line and boundary-trace actions, are alternatively selected according to the sensor information in an on-line manner. Thus even if the automaton does not know any information about obstacle shape and location in the world, the automaton can avoid some unfamiliar obstacles opportunely.

However since the sensor information is to be local one in the world, the algorithms make a deadlock in the world if and only if the automaton does not catch some adequate point L around a tracing obstacle. In 1987, a sensor-based path-planning algorithm was firstly presented by Lumelsky [1,2]. In this algorithm, the leave point L is selected under the following three rules: 1) The automaton can go straight from the leave point L to the goal without any collision; 2) The leave point L is closer than the previous hit point H toward the goal: 3) The leave point L is located on the segment between the start and goal points. Then in 1990, one of the authors presented a sufficient condition to design flexibly the family of sensor-based path-planning algorithms [3-5]. In the algorithm family, selection rules of the leave point are defined as follows: 1) The automaton can go straight from the leave point L to the goal without any collision; 2) The leave point L comes close to the goal asymptotically.

The first rule is permanently requested in a sensor-based path-planning algorithm because the automaton does not trace an unfamiliar obstacle without pushing the obstacle excessively according to sensor information. Thus both previous algorithms keep this rule as the basic one in the selection of the leave point. Moreover in both algorithms, the second rules ensure the convergence of the automaton toward the goal. If they are always satisfied in the world, all obstacle boundaries where the automaton can touch decrease monotonously and asymptotically, respectively in the Lumelsky's and my algorithms. Thus in both ones, the automaton arrives at the goal surely. In addition, the third rule restricts the boundary to only a point, in which the leave point can be picked up by the Lumelsky's algorithm. Thus even if the restricted point is passed over only in the third rule under the influence of the position error, the automaton does not arrive at the goal forever in the Lumelsky's algorithm.

Roughly speaking, the smaller the number of selection rules is, the easier selection of the leave point is. In other words, if many rules are prepared in the selection, an adequate point is easily overlooked if even one of them is spoiled by the position error. Moreover some rule based on the distance between the present and goal points is strongly damaged by a recognition error of the present point. On these observations, we design a robust sensor-based path-planning algorithm of the automaton for some self-positioning error. In the proposed algorithm, only the basic rule is used in selection of the leave point. Owing to the basic rule, the algorithm ensures that the automaton is naturally guided by boundary of an unfamiliar obstacle without pushing it too much with the help of sensor information. Since the basic rule is universally requested for designing a sensor-based path-planning algorithm, the proposed algorithm is regarded as the simplest sensor-based path-planning algorithm.

Because of the algorithm's simplicity, the algorithm frequently picks up several loops in the 2-d world. If the automaton goes in cycles in one of them, the loop becomes a deadlock and consequently convergence of the automaton toward the goal is not satisfied in the world. To overcome this problem, we use some topological properties of the world. By using several types of the properties, the loops are successively smaller as long as they appears in the world. In result, the automaton catches the goal after it gets away from the smallest loop in the world. As compared with the previous distance rule, the topological rule is not affected by the position error and thus convergence of the automaton toward the goal is kept well in the proposed algorithm.

In the section II, an uncertain 2-d world and a set of its important points are defined. The points are uniquely determined on the basis of world topology and the goal, and they are to be robust for

0-7803-0823-9/93/$3.00 (C) 1993 IEEE

some self-positioning error. Each of them is easily recognized by a real-time manner in an uncertain world and thus the proposed algorithm becomes an on-line path-planning algorithm. In the section III, a robust sensor-based path-planning algorithm for the position error is explained, and in the section IV, its convergence of the automaton toward the goal is shown. Moreover in the section V, we illustrate how is the automaton's convergence by the position error. Finally in the section VI, several simulation results are illustrated to ascertain the algorithm's feasibility and to show the convergence in many kinds of uncertain worlds.

II. Our Algorithm Concept Based On An Automaton And Its 2-D Uncertain World

In general, a sensor-based path-planning algorithm leads an automaton to the goal in a 2-d unknown world with a lot of uncertain obstacles. Each obstacle is of free shape. The 2-d world is of finite and thus its boundary is also regarded to be an obstacle. In addition, boundary length of each obstacle is of finite and also the number of obstacles are of finite in the world. Thus an automaton can leave a set of them without tracing the set forever. On the other hand, the automaton is regarded as a point and can pass among some of the obstacles. Also the automaton can act free for all directions and consequently can design every path with free shape in the world.

In this study, the automaton does not know any obstacle shape and location. However the automaton can find some touch against an unfamiliar obstacle via its touch sensor, and therefore the automaton can avoid the obstacle locally on a real-time way according to sensor information. In addition, the automaton knows only a goal in the world, and in the former part of this paper, since the automaton knows the present point exactly, it can calculate the direction toward goal. In the latter part, we investigate how some position error influences convergence of the automaton toward the goal if it does not see the position exactly in the world.

To design a robust path-planning algorithm for some position error, we must ensure the algorithm's convergence without using any distance from the present position. Thus in this study, the robust algorithm is designed as the following feature: the automaton traces an obstacle if it stays in the back boundary, otherwise, i.e., if the automaton stays in the front boundary, it goes straight to the goal (Fig.1). The two boundaries are naturally defined around each obstacle by several types of its points of tangency. Some of them, where the automaton can go straight to the goal without any collision, are called as the external tangential point and are denoted by P_O, and the other of them are called as the internal tangential point and are denoted by P_I in the world. The point P_O is alternatively regarded as one of points P_{OB} and P_{IB} per one of the clockwise and counter-clockwise orders, and also the point P_I is alternatively considered as one of points P_{OF} and P_{IF} per one order. If the automaton moves from the front boundary to the back one, it always passes through the point P_{OB} or P_{IB}. On the other hand, if the automaton moves from the back boundary to the front one, it always passes through the point P_{OF} or P_{IF}.

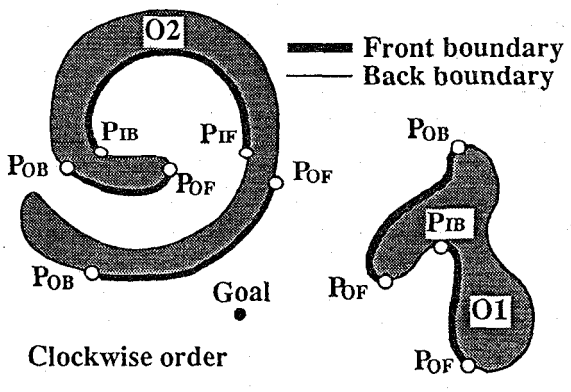


Fig.1 Front and back boundaries and also external and internal tangential points around each obstacle.

In the front boundary, the automaton can go straight to the goal without any interruption of an obstacle, and in the back boundary, the straight-line motion is obstructed by an obstacle. Thus the boundaries are easily identified by checking if the automaton can go straight to the goal or not around an obstacle, and consequently the proposed algorithm becomes an on-line path-planning one. However the simplest algorithm makes a deadlock around a concave obstacle as illustrated in Fig.2. Thus we improve the simplest algorithm as follows: After the automaton passes through the point P_{IF}, the automaton continues to trace the obstacle until it reaches the point P_{OB} or P_{IB} though the boundary between them is to be the front boundary.

In our algorithm, the automaton arrives at the goal finally if it does not meet an already generated path by the same order in the previous tracing. This is the reason that such a path reduces free space in the finite world by a picture drawn with a single stroke of the brush. Otherwise, that is, if the automaton meets an already generated path by the same order in the previous tracing, some loop is generated in the world and also it is divided into two areas by the present loop. In this case, the area with the goal is called as the goal one (Fig.3). Because the goal area is also of finite, the automaton arrives at the goal finally if it does not meet a present loop or an already generated path in the present loop by the same order in the previous tracing. Otherwise an another loop is generated in a present loop and it is registered as a present loop.

If the automaton misses finding the present loop in the world, it makes a periodic motion along the loop as a deadlock and consequently cannot arrive at the goal forever. To overcome this problem, the automaton must recognizes the present loop as soon as possible in the unknown world. For this purpose, it is desirable that the automaton memorizes perfectly the already generated path. However this requires much storage for memorization of infinite points on the already generated path. Thus in this research, the automaton keeps all hit and leave points on the present loop as the active points into the storage 1 and 2, respectively.

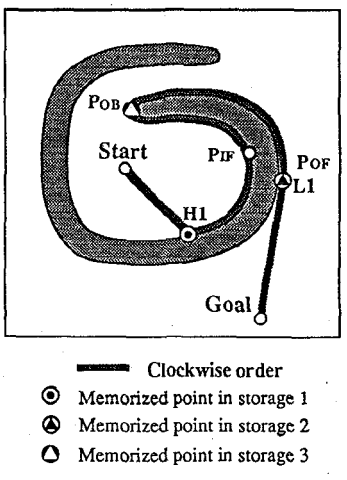


Fig.2 Deadlock prevention around each concave obstacle.

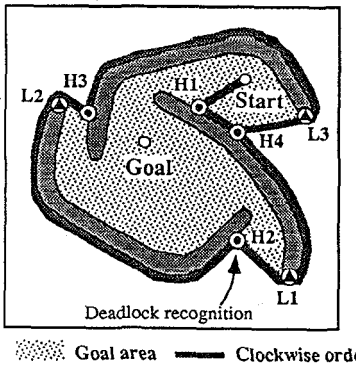


Fig.3 A delayed recognition of a loop by a set of hit points.

If the automaton passes through one of the active points, the algorithm must direct an another action to prevent some periodic motion. In addition to this, if the automaton traces the obstacle at the active hit and leave point by the opposite and the same orders of the previous tracing, respectively, it can enter into the present loop. Then the goal area decreases monotonously in the world and consequently the automaton arrives at the goal (Fig.4). In result, the automaton anyway tries to check the possibility of the active point. Then if the active point lost the possibility in the world, it is converted to the passive one as follows: The hit point in the storage 1 is eliminated and the leave point in the storage 2 is moved to the storage 3. The automaton cannot leave the obstacle from some leave point registered in the storage 3.

In general, if the automaton reaches a point P_O as P_{OF} in an order after tracing it as P_{OB} in the opposite order, the automaton must not leave the obstacle from the point P_{OF}. This is because the automaton does not go out of the present loop. To solve this problem, the algorithm registers the point P_O into the storage 3 in order for the automaton not to leave the obstacle. Even if the automaton finds the point P_O in the storage 3, it continues to trace the obstacle and consequently the algorithm's convergence is ensured (Fig.4).

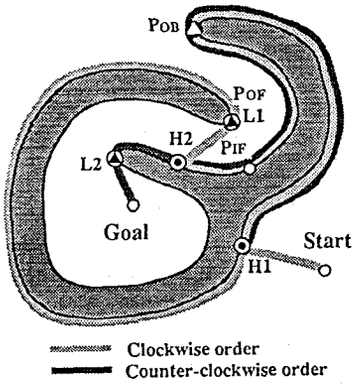


Fig.4 A path-planning example around a complex obstacle

III. A New Sensor-Based Path-Planning Algorithm Based On World Topology

In this section, we explain our sensor-based path-planning algorithm. The algorithm is designed by using a loop reduction explained previously. In this algorithm, the automaton memorizes all pairs of a leave point L_i and its order OL_i, and also memorizes all pairs of a hit point H_i and its order OH_i while tracing some boundary of the obstructive obstacle. If the automaton returns to some point H_i by the order OH_i or returns to some leave point L_i by the opposite one of the order OL_i, the automaton can see a present loop in the 2-d world by a real-time operation.

Thus the algorithm gives the automaton the opposite order to trace the obstacle from the hit point H_i or gives it the same order to trace the obstacle from the leave point L_i. According to the tracing orders, the automaton always enters into the loop and also its goal area decreases. Moreover the automaton must not go out of the loop and also the goal area. Thus in this algorithm the automaton must not leave the obstacle from a point P_O if it reaches the point P_O as P_{OF} after it passes through the point P_O as P_{OB}. When the automaton passes through at the point P_O as P_{OB}, the point is memorized in the storage 3.

The proposed sensor-based path-planning algorithm is indicated as follows:

1. The number i is set by one, i.e., $i=1$, and then the first leave point L_{i-1} is set by the start S, i.e., $L_0=S$. An initial order O is set by the clockwise order C, i.e., $O=C$, and therefore the counter-clockwise order is denoted by CC. If the order $O=C$, its opposite order $\neg O$ is denoted by the counter-clockwise order, i.e., $\neg O=CC$. In addition, all storage 1~3 is initially clear.

2. From the leave point L_{i-1}, the automaton always goes straight to the goal G until one of the following occurs:

(a) The algorithm ends with success if the automaton reaches the goal G.

(b) If the automaton recognizes some contact against an obstacle, it inputs the hit pair of a point H_i and its order OH_i into the storage 1 and then we move to the step 3.

3. The automaton follows the obstacle by the order O and does the following procedures.

(a) If the automaton reaches the goal G, the algorithm finishes with success.

(b) The automaton reaches a point P_O as P_{OF}. In this case, if the point P_O is memorized into the storage 3, the automaton continues to trace the obstacle. Otherwise, the automaton inputs the leave pair of a leave point L_i (the point P_O) and its order OL_i into the storage 2. Then the automaton leaves the obstacle and we move to the step 2.

(c) If the automaton reaches a point P_O as P_{OB}, the automaton memorizes it to the storage 3 and then continues to trace the obstacle.

(d) If the automaton reaches a point P_I as P_{IF}, the automaton traces a back boundary without a break until it catches the point P_{OB} or P_{IB}.

(e) The automaton recognizes some loop if it returns to a hit point H_i in the storage 1 by the same order OH_i. The algorithm selects all hit pairs except the point H_i and all leave pairs on the present loop as the active points. Also the algorithm deletes the other hit and leave points from the storage 1~3. Thus the algorithm does this step again after it resets the order O as the reverse one $\neg O$.

(f) The automaton recognizes some loop if it returns to a leave point L_i in the storage 2 by the opposite one of the order OL_i. The algorithm selects all pairs on the present loop as the active points. Also the algorithm deletes the other hit and leave points from the storage 1~3 and moves the point L_i from the storage 2 to the storage 3. Thus the algorithm does this step again after it resets the order as the reverse one $\neg O$.

(g) If the automaton cannot find any active hit pair in the storage 1, it goes to an active leave point L_i in the storage 2. Then the algorithm does this step again after it resets the order O as the same one OL_i.

(h) If the automaton cannot have any active pair in the storage 1 and 2, the algorithm stops with failure.

The algorithm without the procedure (f) is called as the algorithm Topology1, and also the algorithm with the procedure (f) is called as the algorithm Topology2. In the former algorithm, the automaton destroys the present loop globally in the world, and in the latter algorithm, the automaton destroys the present loop locally in it. Therefore, the latter algorithm is better than the former one for some uncertain world with multiple loops.

IV. Convergence of the Automaton to the Goal

In this section, we ensure theoretically the convergence of the automaton to the goal in the proposed path-planning algorithm. The assurance of the convergence is designed by the following three theorems.

[Theorem 1] If some loop does not appear in an unknown world, the automaton arrives at the goal.

(Proof) If the algorithm does not make any loop in the world, a generated path successively fills in free space of an uncertain world by a picture drawn with a single stroke of the brush. On the other hand, free space is of finite in the robot world. As a result, every free space where the automaton can pass decreases monotonously in the world and the generated path is connected to the goal finally.

[Theorem 2] Even guaranteed that some loop appears in an uncertain world, a present point of the automaton and its goal are separated by an obstacle beside the loop if and only if it does not have any active point. In this case, since the start and goal points are also separated by the obstacle, we can see that there is not any path between the start and goal points initially in the world.

(Proof) If some generated loop does not have any active point, we can find that the automaton traces only an obstacle along the loop as shown in Fig.5. In this case, the loop (including a present point) and the goal are separated by the obstacle and simultaneously the unique world is classified into two areas by the obstacle. Since the automaton joins the loop from the start point, it is always located in the same area with the loop. As a result, the start and goal points are also separated by the obstacle and consequently we can see that there is not any path between them in the world initially if and only if the automaton cannot pick up any active point in the loop.

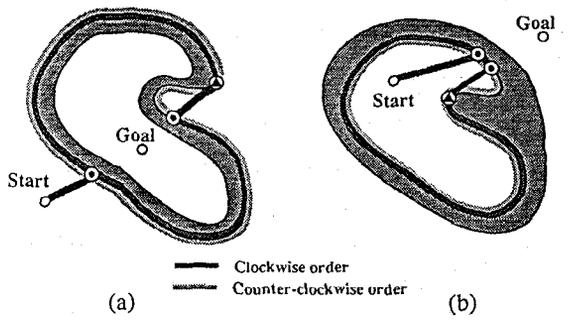


Fig.5 A loop does not have any active point.

[Theorem 3] If some loop has an active point, the automaton can enter into the goal area via the active point. Thus the goal area decreases monotonously in the world as long as there is an active point in the loop. If the monotonous property is always kept, the automaton arrives at the goal finally after leaving from the smallest loop in the world.

(Proof) The automaton can enter into or goes out of the goal area via an active point as shown in Fig.6. The automaton joins the present loop, makes a new loop, or not after the automaton makes an invasion upon the goal area. In the former two cases, the present loop is renewed and consequently the goal area becomes smaller in the world, and in the last case, the automaton arrives at the goal by the same discussion in the Theorem 1 since the goal area is to be finite one. Moreover if the goal area decreases monotonously in the world, the automaton always arrives at the goal because of the finiteness of the world (Fig.7).

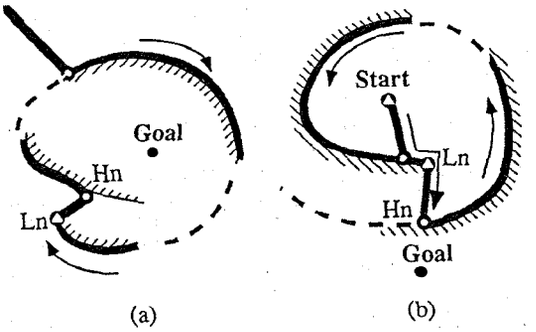


Fig.6 The automaton can enter into or go out of the goal area via each active point.

V. An Influence of Some Position Error For The Convergence

Some impacts in the algorithm's convergence by an error of self-position recognition are indicated in this section. A real automaton always renews its position continuously by an own dead reckoning system. However, a virtual position inside the robot is not usually consistent with a real position in the world, and the difference between the virtual and real positions is named as the position error. In general, the difference grows additionally under the influence of robot mechanism, the quality of floor material and so on. If the automaton allows some position error and nevertheless it arrives at some point near the goal by the error difference, the automaton is regarded as a success to find the goal. This is because the automaton may move from the near point to the goal by some another method.

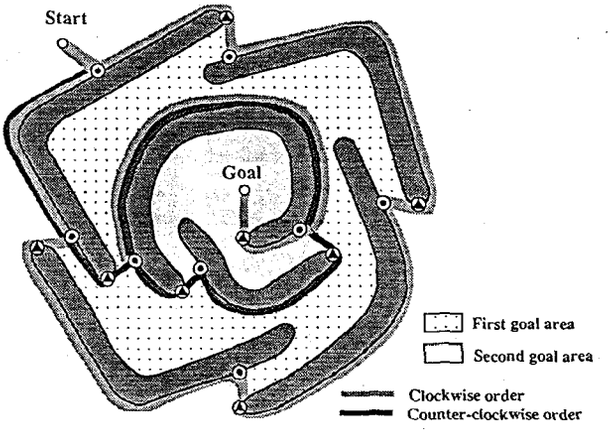


Fig.7 The monotonous reduction of the goal areas in the world.

However the automaton with the error usually cannot identify any point, for example, a hit point H where the automaton determines if some deadlock occurs or not, exactly in the world. Consequently the automaton with some position error may not find any deadlock in all sensor-based path-planning algorithms. To overcome this defective point, sensor-based path-planning algorithms intuitively use the maximum permissible dose of the error as the circle in Fig.8(a). Since all points in the circle are regarded as an important point, the automaton with the error can identify the important point by the circle even though it passes by the side of the important point. However the error circle has some side effects for the algorithm's convergence. Thus we show some aspects of previous sensor-based algorithms by the error circle in this section. The influence is classified into two categories as follows: One is about some local mistake and the other is about some global mistake in the world.

As shown in Fig.8(b), some deadlock is globally made around the goal by the global mistake for finding a reasonable hit point. In this example, the automaton touches an obstacle by a hit point H and then trace the obstacle by the clockwise order. Before the automaton finds only a passage from the start to the goal, the automaton misunderstands that it returns to the hit point H at an arbitrary point A. Consequently the automaton fails in finding a deadlock-free path between the start to the goal in the world at the point A. The global deadlock is kept away from if and only if such a narrow passage is eliminated in the world.

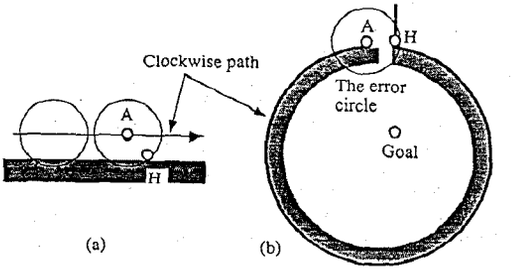


Fig.8 A tough detection of a hit point by the error circle (a) and a wrong detection of a deadlock-free path by the error circle (b).

Moreover if some algorithm adopts a error circle, the automaton cannot identify any pair of points in the error circle. Therefore, if the previous hit point H_i is extremely close to the present leave point L_i in the Lumelsky's algorithm, or if multiple points P_O are to be close each other in our new algorithm, they may make some global deadlock in an uncertain world. However such a problem basically does not occur in my previous algorithm family since the leave point is flexibly selected in a wide boundary.

On the other hand, some deadlock is locally made around an obstacle by the local mistake for finding an adequate leave point. For example, the automaton frequently overlooks the leave point around an obstacle under the influence of the error. In Fig.9, we assume that the error circle is grown after the automaton hits an obstacle by the point H. If the error circle has back boundary of the obstacle as illustrated in Fig.9(a), the basic rule may be spoiled by the error and consequently the adequate leave point may be overlooked in the Lumelsky's algorithm. Even in this case, if the leave point can be selected in a wide boundary, the automaton finds an adequate point as the leave point because the basic rule is kept somewhere in the wide boundary as Fig.9(a). However even in this case, if the distance rule is always not kept in the wide boundary, the automaton may not leave the obstacle forever and consequently may not arrive at the goal surely. On the other hand, the new algorithm eliminates the distance rule and therefore the automaton easily leaves from the front boundary of the obstacle as illustrated in Fig,9(b). In result, the automaton with some position error always find an adequate point as the leave one in a real-time manner according to sensor information and consequently it does not make any deadlock around each obstacle in the new algorithm.

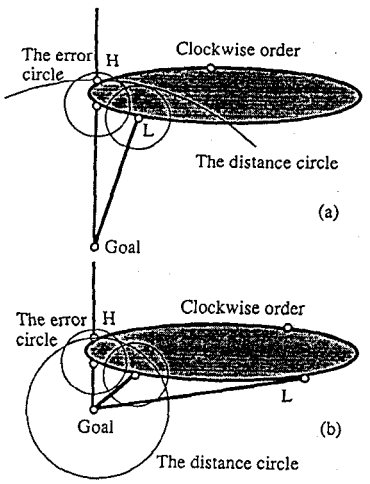
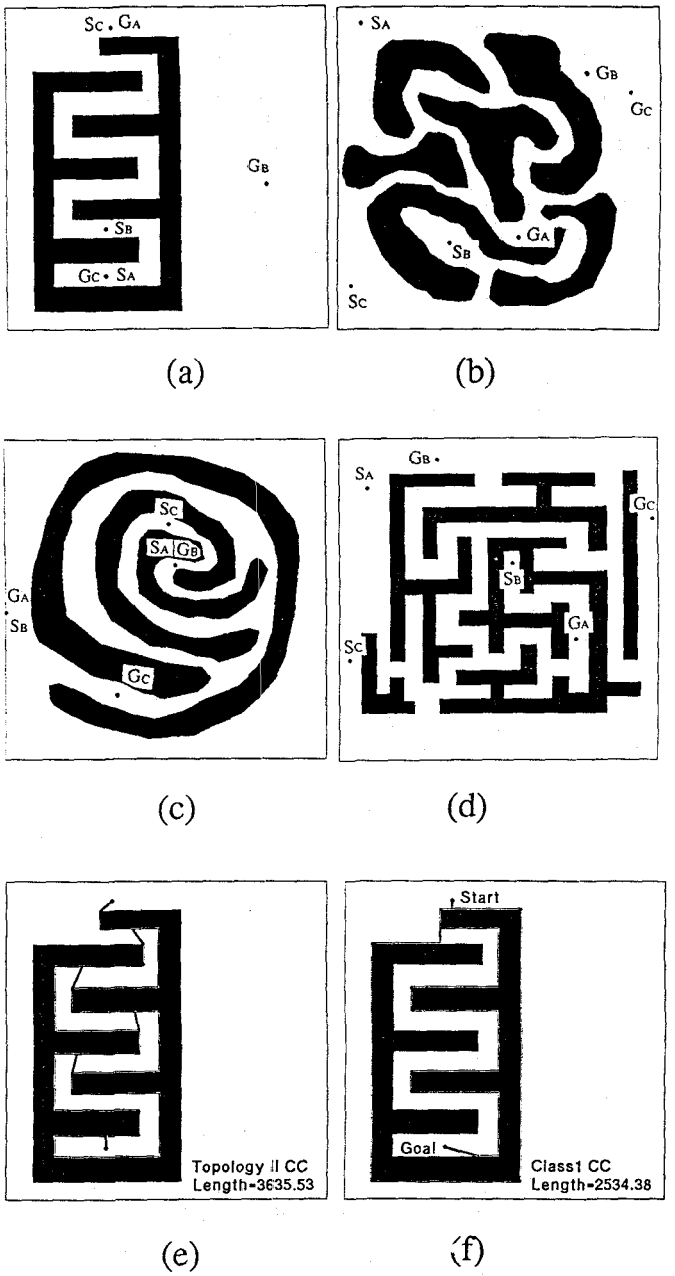


Fig.9 A local deadlock around an obstacle by damaging the basic rule (a) and that around the obstacle by damaging the distance rule (b).



VI. Simulation Results

In this paper, we compare the proposed algorithm with the previous algorithms, i.e., Lumelsky's algorithms Bug1 and Bug2 [1], and our algorithms Class1 ~ Class3 [3]. The algorithms are implemented in the graphics workstation Apollo 3500 under the AEGIS operating system by the C language.

To compare the sensor-based path-planning algorithms, we prepare a simple shape world, two complex shape worlds, and a labyrinthine world, and then locate several kinds of pairs of start and goal points in these worlds as shown in Fig.10(a)~(d). These algorithms make a lot of different deadlock-free paths and we shows their path length by the clockwise and counter-clockwise orders in Table 1. As we can see in these simulation results, the new algorithm frequently generates shorter paths because it does not form similar loops in some of the worlds. Because of the loop reduction in the new algorithm, the automaton finds the minimum loop and consequently catch the goal effectively in the world. In result, the new sensor-based path-planning algorithm is not bad to find some shorter paths in different types of two-dimensional worlds.

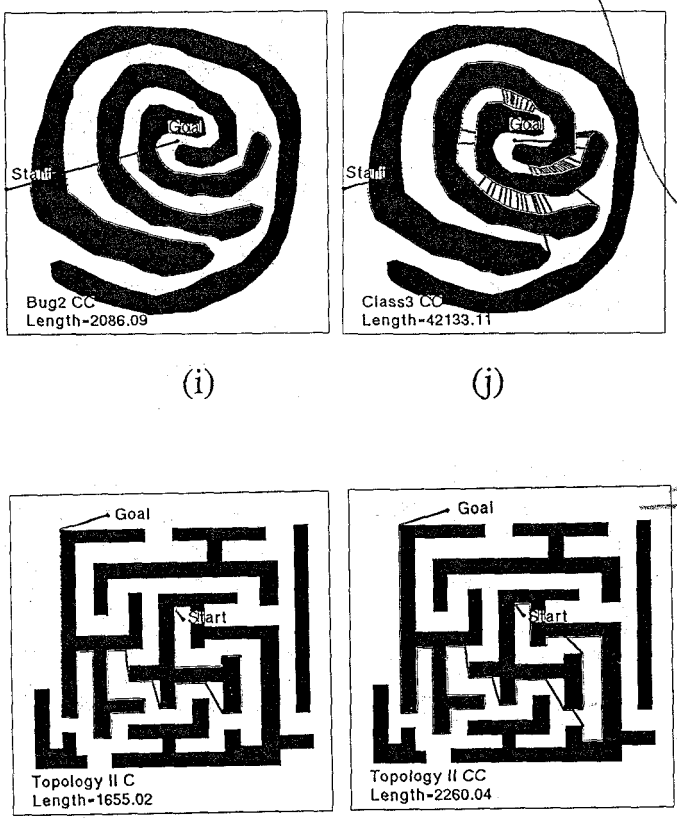


Fig.10 Several pairs of start and goal points in four uncertain worlds (a)~(d) and some of their paths in the worlds (e)~(l).

Table 1 Their path length for the pairs in the four unknown worlds.

	a-A	a-B	a-C	b-A	b-B	b-C
Bug1	5020.11	5111.47	4814.11	2681.35	3677.72	3003.33
Bug2 C	4362.46	1525.09	6307.14	*1064.31	1807.22	1738.02
Class1 C	4092.40	1408.39	*2290.57	2765.98	2269.35	1436.82
Class2 C	4093.79	1408.01	6491.14	2765.90	2270.12	1436.82
Class3 C	18374.70	1411.72	6494.07	2775.93	2123.87	1369.55
Topology1 C	5300.84	*1363.94	7697.02	2760.22	*1554.69	*1231.94
Topology2 C	*3635.53	*1363.94	4357.57	2760.22	*1554.69	*1231.94
Bug2 CC	6707.60	2504.65	6788.97	1131.47	1380.35	1497.33
Class1 CC	6386.20	2463.31	*2534.38	1222.61	1150.10	1266.15
Class2 CC	6385.04	3698.75	7565.86	1220.58	1150.10	1266.15
Class3 CC	39810.83	9842.76	63739.77	1095.62	*1150.06	1278.65
Topology1 CC	4994.52	*2443.98	6473.37	*1080.04	1152.39	*1263.72
Topology2 CC	*3995.55	*2443.98	5682.40	*1080.04	1152.39	*1263.72

	c-A	c-B	c-C	d-A	d-B	d-C
Bug1	7572.24	6373.22	7695.25	5541.50	7049.63	5818.98
Bug2 C	2090.59	4021.40	1590.88	4129.78	2789.97	3138.45
Class1 C	1783.37	*2472.39	1170.66	*2362.01	2588.49	891.27
Class2 C	1780.52	6499.27	1063.69	4043.32	2596.45	890.96
Class3 C	1747.23	84988.47	1053.35	4064.88	1940.12	*889.88
Topology1 C	*1710.37	3895.27	*1053.13	3272.52	*1655.02	890.82
Topology2 C	*1710.37	3895.27	*1053.13	4350.74	*1655.02	890.82
Bug2 CC	4018.90	*2086.09	4359.53	*1507.01	3174.49	4070.23
Class1 CC	3725.10	3789.81	2437.13	2117.13	2829.47	1109.24
Class2 CC	3725.10	3789.77	2437.13	2113.64	2829.47	1117.84
Class3 CC	2419.81	42133.11	3730.47	3739.54	2806.79	1117.84
Topology1 CC	*2322.92	2307.63	*1889.10	5548.12	*2260.04	*1111.62
Topology2 CC	*2322.92	2307.63	*1889.10	2616.52	*2260.04	*1111.62

VII. Conclusions

In this paper, we proposed a new sensor-based path-planning algorithm running in an uncertain 2-d world. In the algorithm, the automaton basically goes straight to the goal, and if it touches some of uncertain obstacles, the automaton traces the obstacle boundary until it can go straight to the goal again. Moreover if the automaton finds some loop in the world, the automaton enters into the goal area. In result, several loops met by the automaton becomes smaller monotonously in the world and then the automaton arrives at the goal surely after it escapes from the minimum loop. The good character is ensured in an online manner.

The algorithm has several good characters as follows. (1) It omits some calculation of the distance from the present point, and therefore it is faster than all previous sensor-based algorithms with the calculation. (2) It is to be robuster than the previous sensor-based algorithms for some self dead reckoning error. The error has a decisive effect for the distance from the present point, and consequently it spoils the previous algorithms. On the other hand, the proposed algorithm is not damaged by the error because its topological property is not relatively spoiled by it. (3) It can generate shorter deadlock-free paths in almost all 2-d worlds as compared with the previous sensor-based algorithms.

References

[1] V.J. Lumelsky and A.A. Stepanov, "Path-planning strategies for a point mobile automaton moving admist unknown obstacles of arbitrary shape," *Algorithmica,* vol.2, pp.403-430, 1987.

[2] V.J. Lumelsky, "Algorithmic and complexity issues of robot motion in an uncertain environment," *J. of Complexity,* Vol.3, pp.146-182, 1987.

[3] H. Noborio, "A path-planning algorithm for generation of an intuitively reasonable path in an uncertain 2D workspace", *Proc. of the USA-Japan Symposium on Flexible Automation,* pp.477-480, 1990.

[4] H. Noborio, "Several path-planning algorithms of a mobile robot for an uncertain workspace and their evaluation," *Proc. of the IEEE International Workshop on Intelligent Motion Control,* Vol.1, pp.289-294, 1990.

[5] H. Noborio, "A sufficient condition to design some sensor-based path-planning algorithm based on the monotonous decrease of the Euclidean distance to the goal," *International Journal of Advanced Robotics,* to appear, 1993.